Lab 5: Static Code Analysis

Name: Neha Rastogi
SRN: PES1UG24AM175
Team: SE_Team_05
Program: restaurant_order_system.py

Objective

Analyze the restaurant order program using Pylint, Bandit and Flake8; document the reported issues and the changes in the corrected program; and reflect on the results.

Before and after results

Tool	Before	After
Pylint	4.81 / 10	10.00 / 10
Bandit	5 findings: 2 High, 1 Medium, 2 Low	0 findings
Flake8	9 diagnostics	0 diagnostics

Evidence and scope

The issue tables use original line numbers from the before reports, checked against restaurant_order_system_original.txt. Fix descriptions are based on the supplied corrected Python file. Locations are not line numbers in the corrected file.

The supplied after reports show no Pylint diagnostics, no Bandit findings and an empty Flake8 report. These are recorded results, not new analysis runs performed for this document. Bandit reports no skipped files or suppressed findings in either run.

The Lab 5 handout requires at least four fixes, an issue table and reflection answers (page 2). Page 3 requires all Bandit findings resolved, Pylint above 9.5, all Flake8 formatting issues resolved, before/after reports and three tool-specific fix commits. The supplied analysis results meet the stated tool targets. Commit history is a separate submission requirement, not established by these reports. The handout does not require screenshots.

Issue table: security findings

All five Bandit findings had High confidence. Severity below is the severity reported by Bandit.

Original line / rule	Issue and severity	Fix in corrected program
26: B110; Pylint W0702; Flake8 E722	Low: bare except followed by pass silently hides errors.	Check whether the table exists before deleting it. Remove the try/except/pass block.
42: B307; Pylint W0123	Medium: eval(promo_expr) executes an arbitrary expression.	Accept only the exact approved promotion string, calculate base_bill * 0.90 directly, and raise ValueError for unsupported expressions.
47: B324	High: MD5 used for a ticket hash is flagged as weak for security use.	Replace hashlib.md5 with hashlib.sha256 and explicitly encode the table ID as UTF-8.
51: B311	Low: random.randint is unsuitable for a security-sensitive POS PIN.	Use secrets.randbelow(9000) + 1000 to generate values from 1000 through 9999.
58: B605	High: os.system constructs a shell command using the filename.	Remove the shell invocation and display the message using print and an f-string. This remains a status message; it does not implement printer integration.

Interpretation

The corrected code removes all five reported Bandit patterns. Replacing eval intentionally restricts the accepted promotions to one approved expression. SHA-256 changes the digest format, and the PIN remains four digits even though its randomness source is stronger.

Issue table: quality and style

Types below are Pylint categories or Flake8 rule types, not Bandit security severities. Related diagnostics are grouped.

Original lines	Reported issue / type	Fix approach
15	W0102: mutable default argument (warning).	Use order_log=None and create a new list inside the function when needed.
15, 23, 30, 40	C0103: function names are not snake_case (convention).	Rename functions and update their call sites to snake_case.
15, 23, 30, 40, 46, 50, 54, 61	C0116: missing function docstrings (convention).	Add docstrings describing each function.
6; 31; 41	W0611 / F401: unused math import; W0612 / F841: unused service_tax and base_bill.	Remove math and service_tax. Use base_bill in the explicit discount calculation.
32; 34	C0121 / E711: comparison to None; R1703 / R1705: redundant conditional and else.	Check table membership, then directly return the negated settlement flag.
18, 19, 75; 83	C0301 / E501: long lines; C0304 / W292: missing final newline.	Split long dictionary entries and calls across lines; add the final newline.
19; 55	C0209: old string formatting; W1514 / R1732: file encoding and resource handling.	Use an f-string; write with a with block and encoding="utf-8".
61; 63	W0621: shadowing orders; W0612: unused loop variable t.	Rename the parameter table_orders and count pending orders using values() and sum().

Reflection answers - proposed questions

The handout requests reflection answers on page 2 but does not list specific questions. The following questions are proposed for this submission, not quoted from the handout. Answers describe the supplied code and reports.

1. What did the three tools reveal, and how did they complement each other?

Pylint identified maintainability and correctness concerns, including a mutable default list, missing docstrings, naming issues and unused values. Flake8 highlighted concise style and code-cleanliness problems such as long lines, unused imports and a bare except. Bandit identified five security-related patterns, including eval, MD5, non-secure PIN randomness and shell execution. Their overlap helped prioritize fixes, while their different checks made the review broader than a single score.

2. Which fixes were most significant, and what did I learn from them?

Removing eval and the shell invocation eliminated two ways that text could be interpreted as executable instructions. I learned to express the required operation directly: the approved discount is calculated arithmetically, and the export status is printed without a shell. I also learned that mutable default arguments can share state between calls, so the order log now starts with None and is initialized inside the function.

3. What do the improved results establish, and what still needs checking?

The saved reports show a Pylint improvement of 5.19 points to 10.00, a reduction from five Bandit findings to zero, and a reduction from nine Flake8 diagnostics to zero. These results show that the reported patterns were addressed under the recorded checks. They do not prove complete security or functional correctness. Functional checks should cover order creation and deletion, missing table IDs, accepted and rejected promotions, pending counts and file export. No separate functional-test evidence was supplied.

Remaining observation

The corrected program still contains a hardcoded POS token, although it now prints only its Boolean presence. A real credential should be provided through an appropriate external configuration or secret store. This was not one of the five reported Bandit findings. Also, hashing a table ID with SHA-256 does not by itself provide ticket authenticity or uniqueness.

Source record

Lab 5: Static Code Analysis student handout, pages 1-3 (Lab_5_Static_Code_Analysis_Student_handout_261020_135935.pdf); restaurant_order_system_original.txt; restaurant_order_system.py; pylint_before.txt and pylint_after.txt; bandit_before.txt and bandit_after.txt; flake8_before.txt and flake8_after.txt. Code and reports were supplied from the student Lab_05 folder.